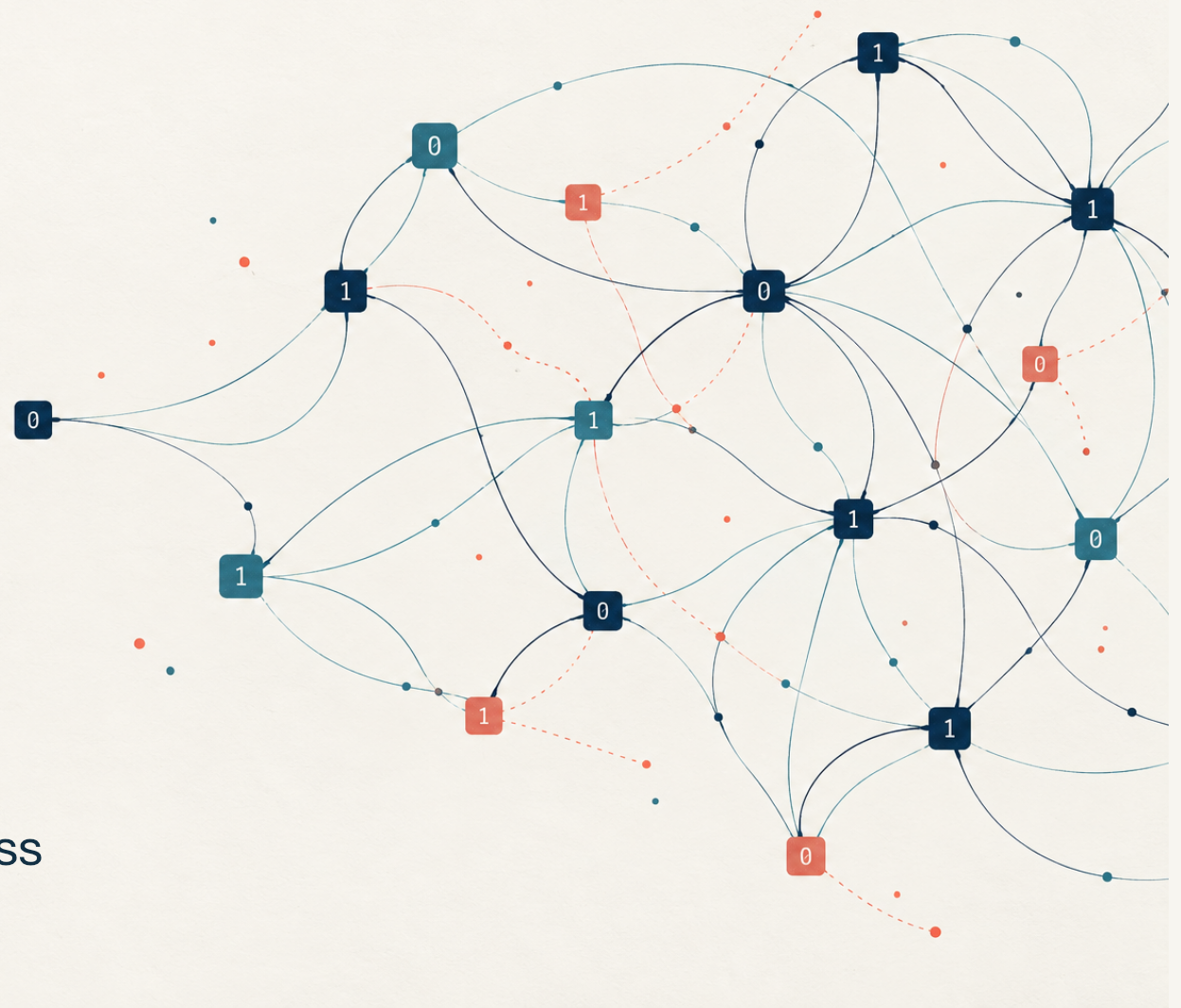


d-LIN

Sparse equations, noisy answers

STUDENT EDITION

A visual introduction to parity, noise and hardness



The central equation

$$\mathbf{b} = \mathbf{M}\mathbf{x} + \mathbf{e}$$

Symbol	Meaning	Size
M	Which variables enter each equation	$m \times n$
x	Hidden binary assignment	$n \times 1$
e	Bits that flip equation outputs	$m \times 1$
b	Observed right-hand sides	$m \times 1$

m counts equations. n counts variables. d counts variables per equation.

Arithmetic with bits

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

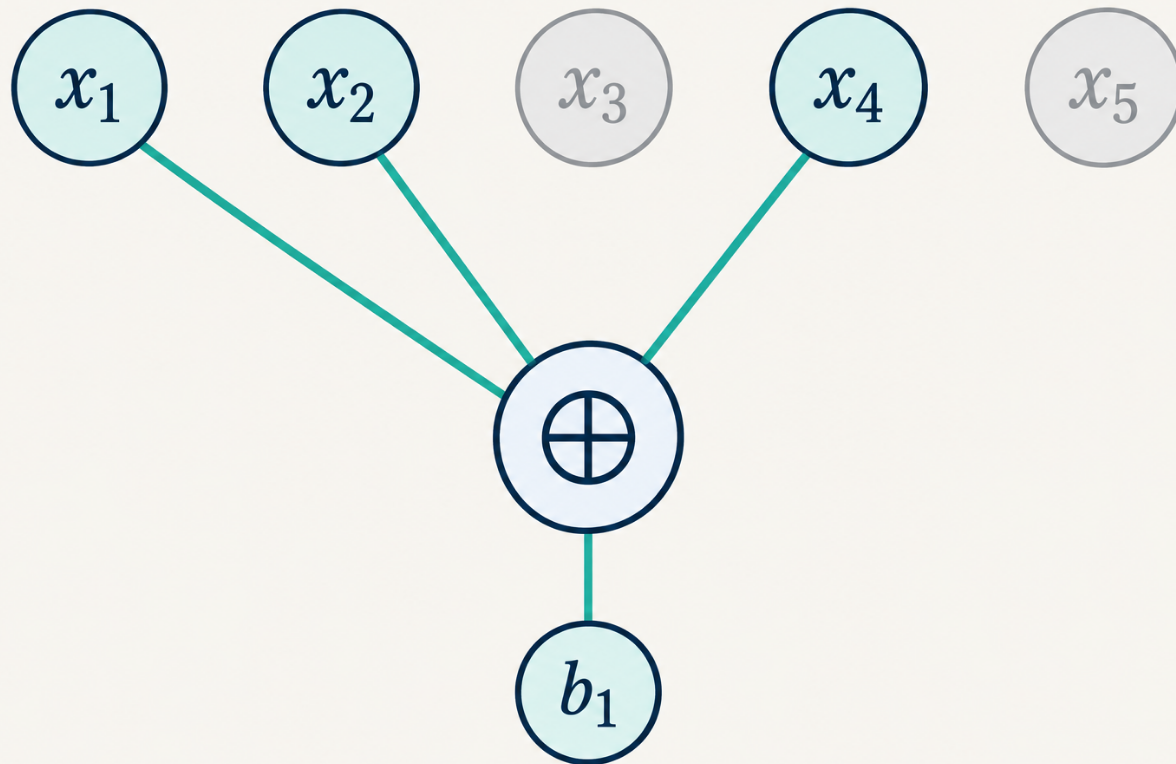
$$1 \oplus 0 \oplus 1 = 0$$

Two ones cancel.
Only the parity of the number of ones matters.

$$a + a = 0$$

All additions in this deck use modulo 2 unless a slide says otherwise.

A sparse equation uses only a few variables



$$d = 3$$

Three selected variables make one parity clause.

$$x_1 \oplus x_2 \oplus x_4 = b_1$$

A 0 in the matrix means that variable does not enter the clause.

The matrix stores every clause

M 4 equations × 5 variables

1	1	0	1	0
0	1	1	0	1
1	0	1	1	0
0	1	0	1	1

x

x_1
x_2
x_3
x_4
x_5

b

b_1
b_2
b_3
b_4

$\times$ $=$

Without noise, $Mx = b$ packages all four equations into one statement.

Exact equations and noisy equations

Exact system

$$\mathbf{M}\mathbf{x} = \mathbf{b}$$

Gaussian elimination finds a solution, or detects inconsistency.

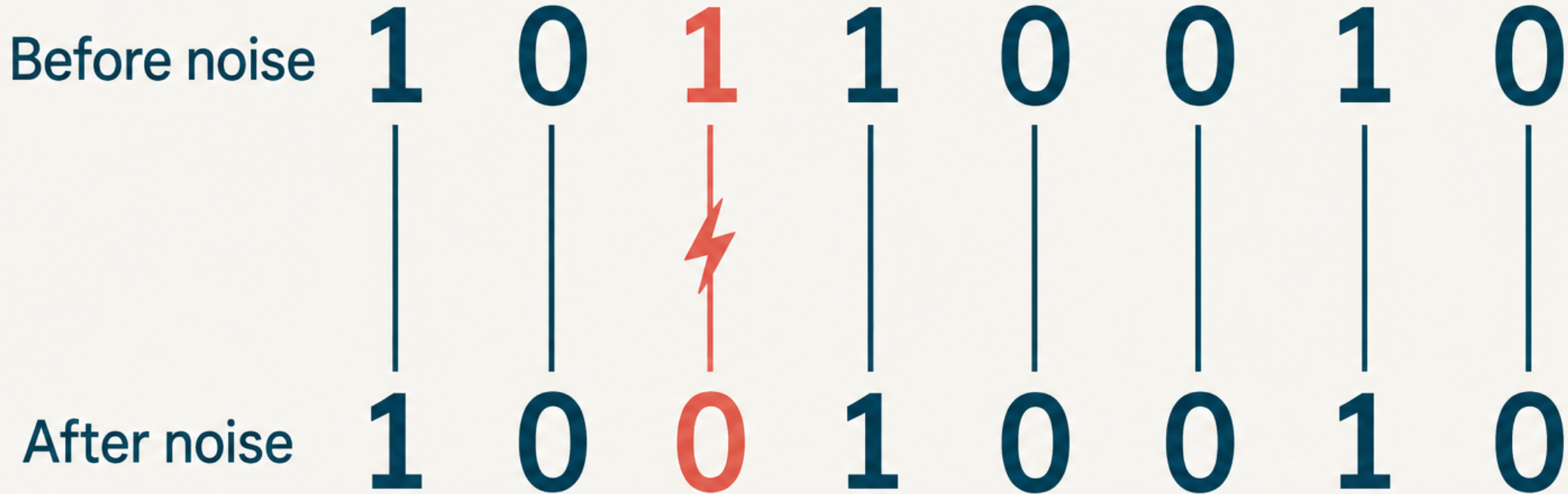
Noisy observations

$$\mathbf{b} = \mathbf{M}\mathbf{x} + \mathbf{e}$$

Some outputs have flipped. Their locations are hidden.

Noise creates an inference problem. Its difficulty depends on the parameters.

Noise flips the output of a clause



$e_i = 0$ unchanged

$e_i = 1$ flipped

A worked example

Take $x = (1, 0, 1, 0, 1)^T$

Clause	Computed Mx	Observed b	Mismatch
$x_1 \oplus x_2 \oplus x_4$	1	1	0
$x_2 \oplus x_3 \oplus x_5$	0	1	1
$x_1 \oplus x_3 \oplus x_4$	0	0	0
$x_2 \oplus x_4 \oplus x_5$	1	1	0

Only the second clause disagrees with the observed right-hand side.

Hamming weight counts the violations

$$\mathbf{Mx} - \mathbf{b} = (0, 1, 0, 0)^T$$

$$\text{wt}(\mathbf{Mx} - \mathbf{b}) = 1$$

75%

3 of 4 clauses satisfied

This assignment has one violation.
A different assignment can do better.

“At least 3/4-satisfiable” is a lower bound for the instance.

Almost satisfiable means few violations

$$\text{wt}(\mathbf{M}\mathbf{x} - \mathbf{b}) \leq \epsilon m$$

An assignment satisfies at least a fraction $1 - \epsilon$ of the clauses.

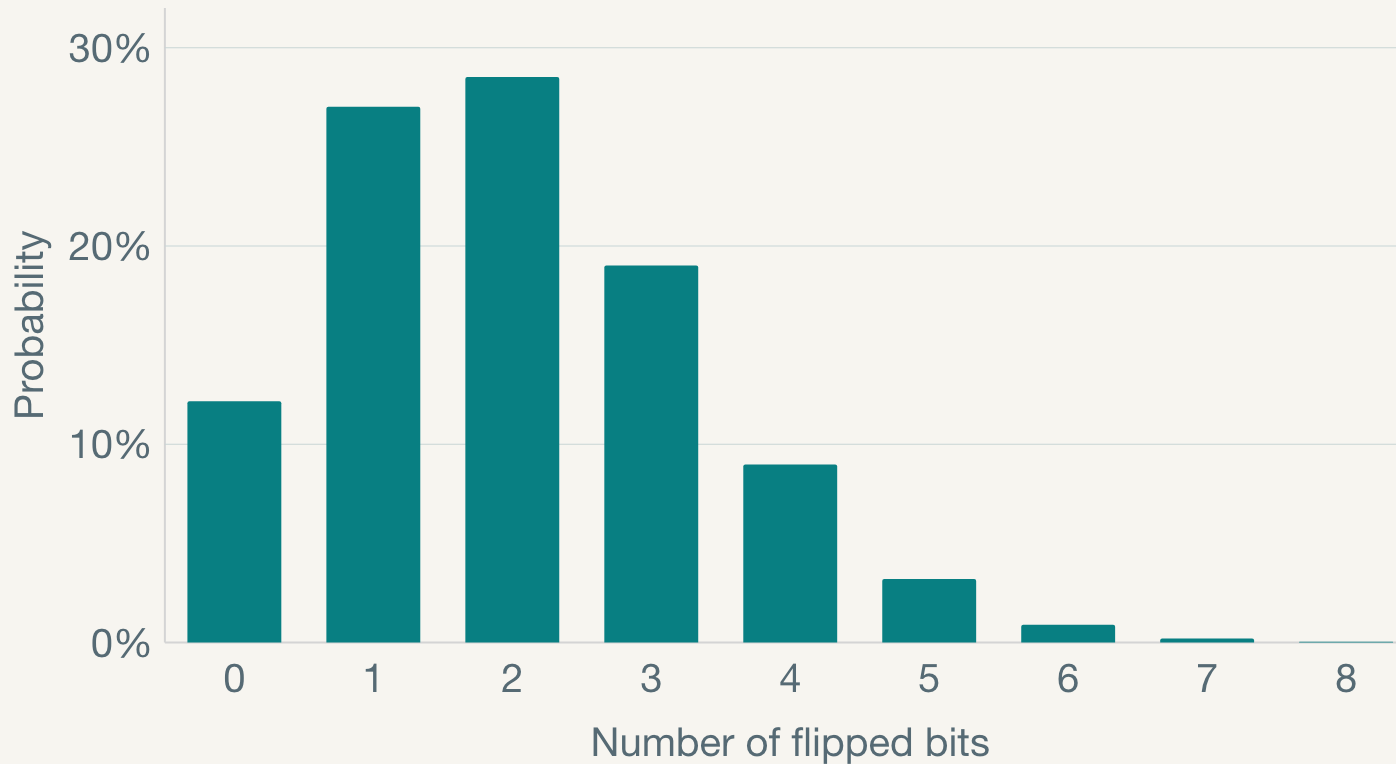
1,000

clauses in total

at most 10

violations when $\epsilon = 0.01$

Bernoulli noise gives an average error count



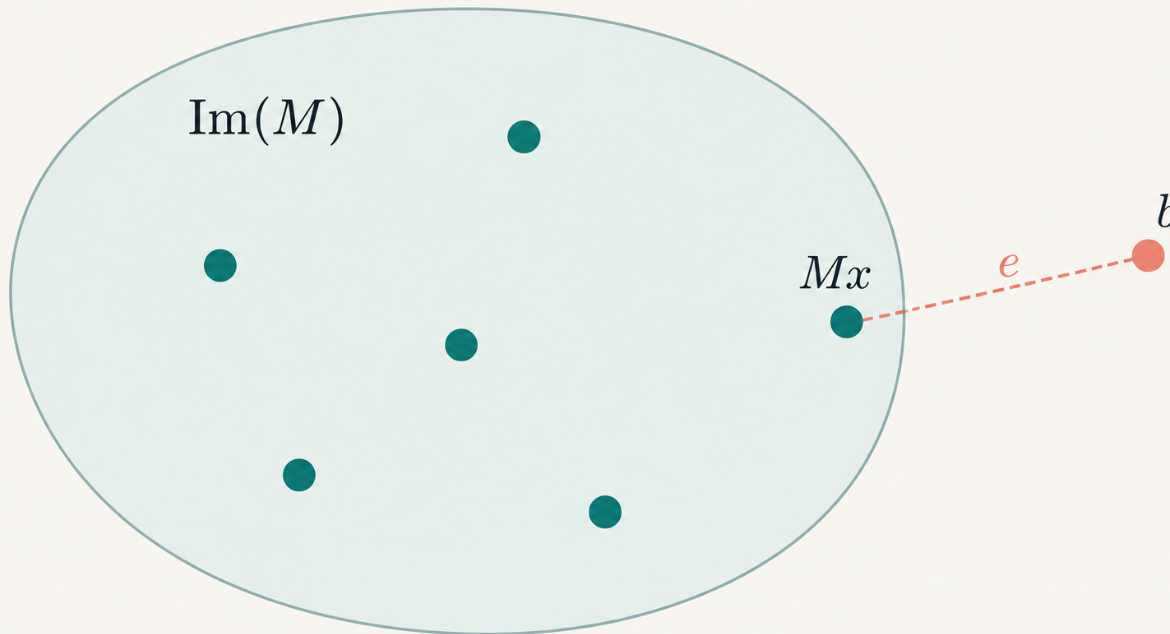
$$\Pr(e_i = 1) = \varepsilon$$

$$\mathbb{E}[\text{wt}(\mathbf{e})] = m\varepsilon$$

20 clauses
10% flip probability
2 errors on average

An average of 2 errors does not guarantee at most 2 errors.

The image of M contains every noiseless output



$$\text{Im}(M) = \{Mz : z \in F_2^n\}$$

Mx is one possible clean output.

b adds the error pattern e .

Hamming distance counts changed bits. The sketch shows one possible noisy observation.

The computational questions

Task	What the algorithm must do
Search3LIN	Recover the original hidden assignment x
Approximate recovery	Recover most bits of x
Decision	Tell noisy parity labels from independent random labels
Residual fitting	Find an assignment violating few equations

A low number of violated equations does not always identify the original assignment.

The hardness assumption

Random instances from a specified distribution

ABW assumes efficient algorithms cannot reliably recover x for particular choices of m , d and ϵ .

Evidence

The paper rules out certain restricted attack families.

Remaining assumption

Hardness against every efficient algorithm remains unproved.

d-LIN, LPN, LWE and Module-LWE

Problem	Linear structure	Error model
d-LIN	Binary rows with exactly d ones	Independent bit flips
LPN	Uniform binary rows, typically dense	Independent bit flips
LWE	Uniform matrices over $\mathbb{Z}_q$	Small centered modular errors
Module-LWE	Polynomial matrices over $\mathbb{R}_q$	Small polynomial coefficients

Changing the matrix distribution or the noise changes the assumption.

LWE and Module-LWE structure

$$\text{LWE: } \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$$

A contains ordinary modular coefficients. Small errors perturb the outputs.

$$\text{Module-LWE: } \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \text{ in } Rq^k$$

A, s and e contain polynomials. Each polynomial equation represents many linked coefficient equations.

Module structure is different from fixing d nonzero bits in each d-LIN row.

What ABW proves about public-key encryption

Theorem 2: one sparse-noise assumption

Hardness of 3-LIN at $m = c n^{1.4}$ and $\varepsilon = n^{-0.2} / c$ implies semantically secure public-key encryption.

Theorem 3: an additional graph assumption

A d-LIN assumption plus hidden unbalanced expansion allows another parameter regime.

Semantic security hides the message from efficient observers.

These are asymptotic existence theorems without a concrete parameter set.

A hidden dependency cancels the assignment

A small cancellation example

$$y_1 = x_1 + x_2 + e_1$$

$$y_2 = x_2 + x_3 + e_2$$

$$y_3 = x_1 + x_3 + e_3$$

$$x_i + x_i = 0$$

Each variable
appears twice.

$$y_1 + y_2 + y_3 = e_1 + e_2 + e_3$$

ABW uses a secret short dependency as part of a public-key encryption construction.

The ABW bit-encryption mechanism

Public key	Secret key
Sparse matrix M	Short set S with $\sum_{i \in S} M_i = 0$ and $m \in S$

$$\text{Enc}(\sigma) = Mx + e + \sigma u_m$$

$$\text{Dec}(c) = \sum_{i \in S} c_i = \sigma + \sum_{i \in S} e_i$$

σ is the message bit. u_m has a 1 only in the last position.
Fresh x and e randomize every encryption.

A short dependency keeps decryption useful

Dec(c) = σ + the parity of r error bits

Example: $r = 10$, $\varepsilon = 0.01$

about 9.15%

raw decryption error

Longer dependencies accumulate noise. ABW uses amplification to obtain reliable encryption.

Reasons to study d-LIN for public keys

A different foundation

ABW connects natural sparse noisy constraints to public-key encryption.

Simple local arithmetic

Core evaluation uses $O(md)$ bit operations. Row index lists use $O(md \log n)$ bits.

A proved conditional construction

Theorems turn specified average-case hardness assumptions into semantic security.

Costs and unresolved issues for public keys

Security assumptions

Parameters and sampling matter.

A short hidden dependency must remain hard to exploit.

Restricted attack evidence leaves a general hardness assumption.

Implementation costs

Noise and dependency length compete with correctness.

Amplification adds overhead.

Concrete parameters, CCA security and quantum analysis need more work.

Cheap sparse XOR operations alone do not establish an efficient final cryptosystem.

ABW and a concrete Module-LWE standard

Aspect	ABW d-LIN construction	ML-KEM
Role	Asymptotic PKE existence result	Standardized key encapsulation
Security statement	Semantic security under stated assumptions	IND-CCA2 goal under Module-LWE and proof conditions
Parameters	No concrete 128-bit set in ABW09	ML-KEM-512 / 768 / 1024
Example sizes	No like-for-like benchmark here	ML-KEM-768: key 1184 B, ciphertext 1088 B

PKE encrypts messages. A KEM establishes a shared secret for a larger protocol.

Reading path and main distinctions

Question	Key distinction
What is the task?	Exact recovery, approximate recovery, decision or residual fitting
What makes it hard?	A precise sampled distribution and an unproved average-case assumption
What does ABW prove?	Semantic-security existence under specific theorem hypotheses
How does it compare?	Same noisy-linear template, different distributions and reductions

ABW09: §§2, 5–7 and 9

Regev on LWE, Langlois–Stehlé on Module-LWE, NIST FIPS 203

Try explaining why exact equations, noisy recovery and public-key encryption are different tasks.